Integración de Servicios Cloud GCP

Supervisión, alta disponibilidad y balanceo de carga

Semana 13 - Sesión 25

Rel Guzman



Revisión: BigQuery sin BigQuery

Qué harás

Las consultas del lab de la sesión anterior, ahora con **Python** y **DuckDB**: un motor SQL que corre dentro de Python y, como BigQuery, lee por columnas. No necesitas cuenta de Google Cloud.

De dónde salen los datos

No se descargan: la primera celda del notebook ejecuta `generar_ventas.py`, que crea `ventas.csv` con **1.2 millones** de pedidos de Pedidos Express (2023–2025). Todos obtienen los mismos datos.

```python
import duckdb

duckdb.sql("""
  SELECT categoria,
    ROUND(SUM(monto), 2) AS total_soles
  FROM ventas
  GROUP BY categoria
  ORDER BY total_soles DESC
""")
```

Cómo empezar

1. En Colab: **Archivo** → **Abrir cuaderno** → **GitHub**.
2. Pega el repositorio del curso:
   `github.com/rgap/rgap-Integracion-de-Servicios-Cloud-Gcp-Agosto2026`
3. Elige `Sesion25/practica/practica_bigquery_python.ipynb`, ejecuta las celdas en orden y resuelve los 5 retos del final.

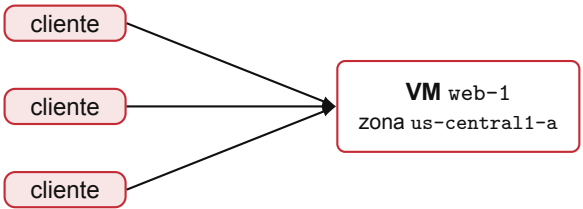
Revisión: compara tus resultados

Pregunta del notebook	Tu resultado
¿Cuántas filas tiene la tabla?	
4.3 ¿Qué categoría tiene más pedidos?	
4.5 ¿Qué categoría vende más dinero?	
Paso 5: ¿cuánto ocupa el CSV? ¿Y el Parquet?	

Responde

1. ¿La categoría con más pedidos es la que más dinero vende? ¿Por qué?
2. ¿Por qué la consulta sobre Parquet termina antes que la consulta sobre CSV?
3. ¿Qué tienen en común DuckDB y BigQuery?

Revisión: la pregunta de hoy



Viernes, 20:00.
Hora de la cena.
Llegan 10 veces más pedidos.
La página no carga.

1. Supervisar	2. Alta disponibilidad	3. Balanceo de carga
¿Cómo te enteras **antes** de que llamen los clientes?	Si `web-1` se apaga, ¿qué sigue respondiendo?	Si pones 3 VMs, ¿cómo se reparten los clientes entre ellas?

Logro de la sesión

Al finalizar, podrás explicar qué es **supervisar recursos** y qué ofrece **Google Cloud Observability**, crear en **Cloud Monitoring** un panel, una alerta y una verificación de tiempo de actividad, y explicar cómo la **alta disponibilidad**, el **balanceador de carga** y **Cloud CDN** mantienen una aplicación en línea.

Saberes previos

- Si una página web tarda 10 segundos en cargar, ¿qué podría estar pasando en el servidor?
- Creaste una VM en la zona `us-central1-a`. ¿Qué le pasa a tu página si esa zona tiene una falla?
- Tienes tres VMs con la misma página y cada una tiene su propia IP externa. ¿Qué dirección les das a los clientes?

Utilidad de esta sesión

Toda aplicación en producción falla alguna vez: se llena un disco, la CPU llega al 100 %, una VM se reinicia. Si sabes **supervisar**, te enteras por una alerta y no por un cliente. Si sabes repartir la aplicación entre varias VMs detrás de un **balanceador**, una falla no deja a nadie sin servicio.

Parte 1: supervisar recursos

Saber qué pasa en tus servidores antes de que te lo digan los clientes

¿Qué es supervisar un recurso?

Definición

Medir de forma continua el estado de los recursos (VMs, bases de datos, aplicaciones), guardar esas medidas y **avisar** cuando algo sale de lo normal.

Métricas

Números que cambian en el tiempo

```
web-1 · uso de CPU
  20:00   35 %
  20:01   62 %
  20:02   95 %
```

Registros

Líneas de texto con lo que pasó (*logs*)

```
20:02:14 GET /menu 200
20:02:15 GET /pagar 500
    20:02:15 ERROR: no se
  pudo conectar a la base
```

Alertas

Reglas que avisan a una persona

Si el uso de CPU pasa de **80 %** durante **5 min**, envía un correo al equipo

Cada pieza responde una pregunta

Métricas: **¿cuánto?** Registros: **¿qué pasó exactamente?** Alertas: **¿a quién aviso?**

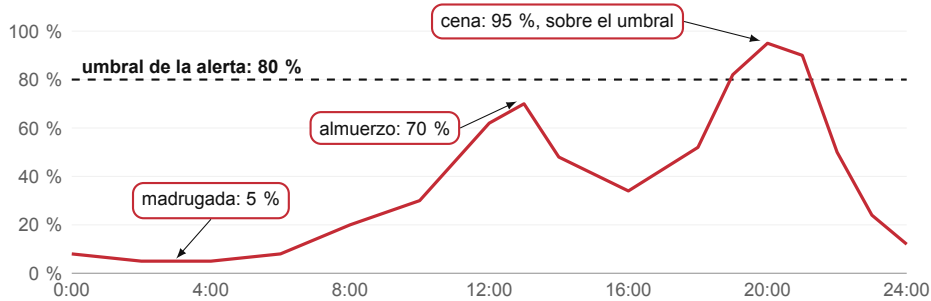
Las métricas que más se revisan

Métrica	Qué mide	Señal de problema
Uso de CPU	Qué porcentaje del procesador se usa	Se queda sobre el 80 % por varios minutos
Memoria	Cuánta RAM ocupa la aplicación	Llega cerca del 100 %: la aplicación se cierra
Disco	Espacio usado del disco	Pasa del 90 %: pronto no se podrá guardar nada
Red	Bytes que entran y salen de la VM	Cae a 0 en horario de atención
Latencia	Cuánto tarda en llegar una respuesta	Sube de 200 ms a 3 s
Errores	Respuestas con código 5xx (falla del servidor)	Aparecen muchas de golpe

Dato útil

Una VM envía sola su uso de CPU, red y lecturas de disco. Para ver la **memoria** y el **espacio de disco** se instala en la VM el **Agente de operaciones** (*Ops Agent*).

Leer una gráfica de uso de CPU



Lo normal

La CPU sube en hora punta y baja de madrugada. Un pico corto no es una falla.

Lo raro

Que se quede alta mucho tiempo, que esté alta sin clientes o que la línea desaparezca (la VM no envía datos).

Problemas que ocurren seguido

Qué pasa	Qué ves	Qué se hace
Hora punta: muchos más pedidos	CPU al 95 % y latencia alta	Agregar VMs (Parte 3)
Los registros llenan el disco	Disco al 100 % y errores al guardar	Borrar registros viejos o agrandar el disco
La VM se detuvo o se reinició	La verificación de tiempo de actividad falla; la gráfica se corta	Revisar los registros y tener otra VM (Parte 2)
Se subió una versión con un error	Errores 5xx justo después de la hora del cambio	Volver a la versión anterior
Un proceso quedó trabado	CPU al 100 % a las 4 a. m., sin clientes	Revisar el proceso y reiniciarlo

El orden de trabajo

La **alerta** avisa → la **métrica** dice desde cuándo y cuánto → los **registros** dicen por qué.

Google Cloud Observability

Definición

Conjunto de servicios de Google Cloud para supervisar tus recursos. Ya viene activado en cada proyecto: las VMs, Cloud SQL y los buckets envían métricas y registros sin que instales nada.



Cloud Monitoring (lo usas en el lab)	**Cloud Logging**
¿Cuánta CPU usó `web-1` entre 19:00 y 21:00?	¿Qué error dio la página `/pagar` a las 20:02?

Cloud Monitoring: lo que harás en el lab

Panel (*dashboard*)
Una página con gráficos que tú eliges. En el lab: `My dashboard` con un gráfico de líneas del uso de CPU.

Explorador de métricas
Ver una métrica al momento, sin guardarla en un panel.

Política de alertas
Una regla: métrica + umbral + a quién avisar. En el lab: `My Alert Policy`.

Grupo
Junta recursos por su nombre para verlos juntos. En el lab: las VMs cuyo nombre contiene `nginx`.

Verificación de tiempo de actividad (*uptime check*)
Google pide tu página por HTTP cada minuto desde varias partes del mundo. Si no responde, te avisa. En el lab: `My Uptime check` sobre el grupo.

El lab ya trae tres VMs con un servidor web: `nginxstack-1`, `nginxstack-2` y `nginxstack-3`.

Cómo funciona una política de alertas



En el lab

La política tiene **dos condiciones** y se dispara solo si se cumplen todas (*All conditions are met*). El umbral es 20 % para que la veas dispararse; en producción se usa algo como 80 %.

Elegir el umbral

Muy bajo: llegan correos todo el día y el equipo deja de leerlos. Muy alto: avisa cuando los clientes ya se fueron.

Al terminar el lab

Desactiva la política (tarea 6). Las alertas siguen activas un tiempo después de borrar el proyecto.

Parte 2: alta disponibilidad

Que la aplicación siga en línea aunque falle una pieza

¿Qué es la disponibilidad?

Disponibilidad

El porcentaje del tiempo en que el servicio responde. Un año tiene $365 \times 24 = 8\,760$ horas.

Disponibilidad	Caída al año	Caída al mes	Ejemplo
99 %	87.6 horas	7.3 horas	Una VM sola que a veces se reinicia
99.9 %	8.8 horas	44 minutos	Varias VMs en una zona
99.99 %	53 minutos	4.4 minutos	Varias VMs en varias zonas

Cálculo: 1 % de 8 760 horas = 87.6 horas. Los ejemplos son orientativos.

Alta disponibilidad

Diseñar el sistema para que la falla de **una pieza** (una VM, un disco, una zona) no apague el servicio.

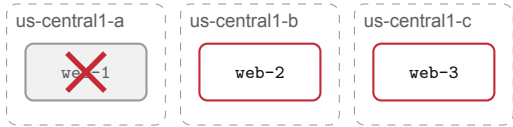
Punto único de falla

Una VM en una zona



Falla `web-1` o la zona:
0 servidores, página caída

Tres VMs en tres zonas



Falla la zona `a`:
quedan 2 servidores, la página sigue

Punto único de falla

Una pieza que, si falla, apaga todo el sistema. Para quitarlo: **duplicar** la pieza y poner las copias en **zonas distintas** de la región.

Alta disponibilidad en Google Cloud

Pieza	Sin alta disponibilidad	Con alta disponibilidad
Servidores web	Una VM en una zona	**Grupo de instancias administrado**: varias VMs iguales, creadas desde una plantilla, en varias zonas. Si una VM se detiene, el grupo la vuelve a crear.
Base de datos	Cloud SQL en una zona	**Cloud SQL con alta disponibilidad**: una copia en espera en otra zona. Si la principal falla, la copia toma su lugar sola.
Archivos	Un disco en una VM	**Cloud Storage**: guarda varias copias de cada archivo sin que hagas nada.

La regla

Más de una copia, en zonas distintas, y algo que detecte la falla y cambie solo.

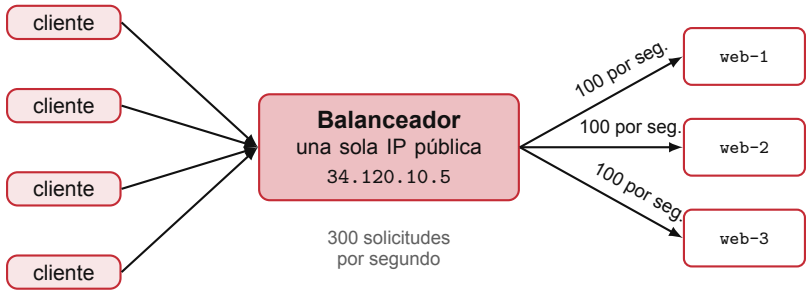
El costo

Tres VMs cuestan tres veces una. Se elige según cuánto le cuesta al negocio una hora caída.

Parte 3: balanceo de carga y CDN

Una sola dirección para muchos servidores

El balanceador de carga



Balanceador de carga

Servicio que recibe todas las solicitudes en una sola dirección IP y las **reparte** entre varios servidores. Los clientes no saben cuántas VMs hay detrás.

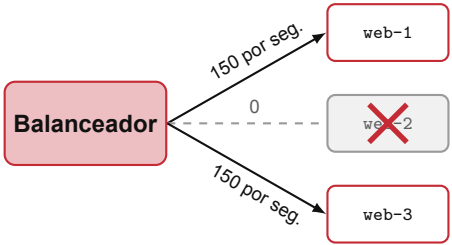
Sin balanceador

Tres IPs distintas. Si publicas la de `web-1`, las otras dos no reciben a nadie.

Con balanceador

Una IP. Cada VM atiende una parte; si agregas una cuarta, el balanceador la usa.

Verificación de estado: no enviar tráfico a una VM caída



Hora	Pregunta a `web-2`	Estado
20:00:00	`GET /` → 200 OK	sana
20:00:05	`GET /` → sin respuesta	sana (1 falla)
20:00:10	`GET /` → sin respuesta	**no sana**

Valores por defecto: pregunta cada 5 s; 2 fallas seguidas la marcan como no sana.

Verificación de estado (*health check*)

El balanceador pregunta a cada VM cada pocos segundos si responde. Si no responde, deja de enviarle clientes; cuando vuelve a responder, la usa otra vez.

Verificación de estado (balanceador)	**Verificación de tiempo de actividad** (Monitoring)
Decide sola a qué VM enviar tráfico	Avisa a una persona que algo no responde

Tipos de balanceador

	De aplicaciones	De red
Protocolo	HTTP y HTTPS	TCP, UDP y TCP con SSL
Qué mira	La URL de cada solicitud	Solo la IP y el puerto
Qué puede hacer	Enviar `/api` a un grupo de VMs y `/imagenes` a un bucket; manejar el certificado HTTPS	Repartir conexiones sin abrir su contenido
Ejemplo	La página y la app de Pedidos Express	Un servidor de juegos o de video en vivo

Externo

Recibe tráfico de Internet. Es el que ven los clientes.

Interno

Reparte tráfico entre servicios dentro de tu red VPC. No tiene IP pública.

Autoescalado: más VMs cuando hace falta

Autoescalado

El grupo de instancias agrega o quita VMs según una métrica. Usa la misma métrica que ves en Cloud Monitoring.

Configuración de Pedidos Express

Mínimo **2** VMs, máximo **6**.
Objetivo: uso de CPU promedio de **60 %**.

Hora	CPU promedio	VMs
04:00	10 %	2 (mínimo)
13:00	75 % → 58 %	2 → 3
20:00	90 % → 60 %	3 → 5
23:30	20 %	5 → 2

Cómo encaja con el balanceador

El balanceador envía tráfico al grupo. Cuando el grupo crea una VM nueva, el balanceador empieza a usarla apenas pasa la verificación de estado.

Cloud CDN: copias cerca del cliente



Primera solicitud

La caché no tiene `logo.png`: lo pide a las VMs, se lo entrega al cliente y **guarda una copia**.

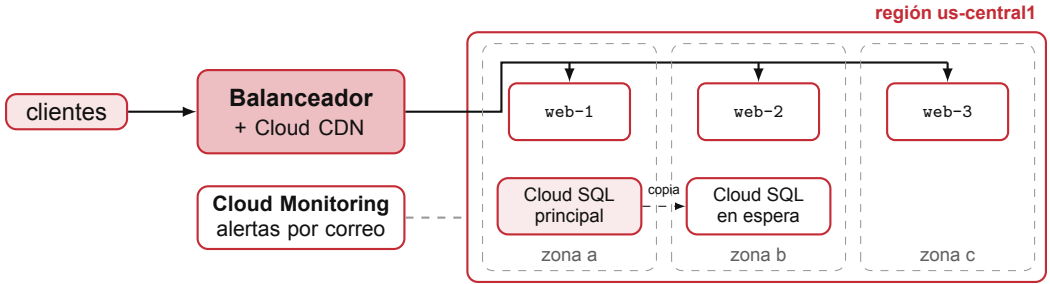
Siguientes solicitudes

La caché responde con su copia. La solicitud **no llega** a las VMs.

Sí va en la CDN (igual para todos)	**No va en la CDN** (cambia por cliente)
Fotos de productos, logo, CSS, JavaScript	Carrito, pedidos de Ana, estado del pago

Cloud CDN se activa con una casilla en el balanceador de aplicaciones externo.

Todo junto: Pedidos Express en línea

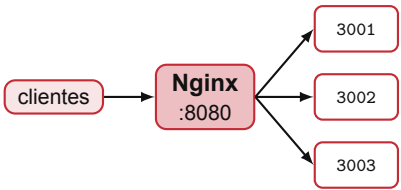


Si pasa esto…	…responde esto
Falla `web-1`	La verificación de estado la saca; `web-2` y `web-3` atienden
Llega la hora de la cena	El autoescalado agrega VMs; la CDN entrega las fotos sin tocar las VMs
Falla la base principal	Cloud SQL pasa a la copia en espera de la zona `b`
Algo sale de lo normal	Cloud Monitoring envía un correo al equipo

Práctica 1: balanceo con Nginx en tu computadora

Caso de estudio

Pedidos Express prepara su tienda online para el Cyber Day: tres copias del servidor (Node.js) detrás de **Nginx**, que reparte por turnos: 3001, 3002, 3003, 3001… (*round robin*).



Ruta	Qué hace	El servidor…
`GET /`	Página principal	responde al instante
`GET /productos`	Consulta la base de datos	**espera** 2 s
`POST /comprar`	Cobra con el banco	**espera** 5 s
`GET /reporte`	Calcula un reporte	**trabaja** 5 s con la CPU

Cómo empezar (carpeta `Sesion25/practica_balanceo/`)

1. Tres terminales, un servidor en cada una: `node server.js 3001` (y 3002, 3003)
2. Otra terminal, para encender Nginx: `nginx -p "$PWD" -c nginx.conf`
3. Otra terminal, para simular clientes: `node prueba.js / 6 --en-fila`

Práctica 1: seis pruebas

#	Comando	Qué observar
1	`node prueba.js / 6 --en-fila`	¿En qué orden responden los servidores?
2	`node prueba.js /productos 6 --directo 3001`	Un solo servidor: ¿cuánto tarda en total?
3	`node prueba.js /reporte 6 --directo 3001`	Un solo servidor: ¿cuánto espera el sexto cliente?
4	`node prueba.js /reporte 6`	Con Nginx: ¿cuánto tarda en total?

5	`node prueba.js /reporte 1`
	y, antes de 5 s, en otra terminal:
	`node prueba.js / 6`
		¿Qué visitas tardan? ¿A qué servidor le tocaron?

6	Apaga el 3002 (`Ctrl+C`) y luego: `node prueba.js / 6 --en-fila`	¿Algún cliente ve un error? ¿Qué servidores responden?

La idea

Cuando el servidor **espera**, atiende a muchos a la vez. Cuando **trabaja** con la CPU, los clientes hacen cola: ahí ayuda repartir entre varios servidores.

Práctica 2

Realiza el laboratorio

Supervisa recursos con Google Cloud Observability

`skills.google/catalog_lab/1785`

Al terminar

Tienes un panel con el uso de CPU de las VMs `nginxstack`, una política de alertas con dos condiciones, el grupo `Instancias de VM` y una verificación de tiempo de actividad.

No olvides

Desactivar la política de alertas antes de finalizar el lab (tarea 6).

Cierre: preguntas para comprobar

Responde con tus palabras

1. ¿Qué diferencia hay entre una métrica y un registro? Da un ejemplo de cada uno.
2. La CPU de una VM está al 100 % a las 4 a. m., sin clientes. ¿Qué podría estar pasando y dónde lo revisas?
3. ¿Qué es un punto único de falla? ¿Cómo lo quitas en los servidores web de Pedidos Express?
4. ¿Qué hace el balanceador cuando una VM no pasa la verificación de estado?
5. ¿Qué archivos de Pedidos Express conviene entregar con Cloud CDN y cuáles no? ¿Por qué?

Idea final

Supervisar te dice qué pasa. La alta disponibilidad y el balanceador hacen que una falla no deje a los clientes sin servicio.

Referencias

- Laboratorio: *Supervisa recursos con Google Cloud Observability* (*Monitor Resources with Google Cloud Observability*), curso *Essential Google Cloud Infrastructure: Core Services*: `skills.google/catalog_lab/1785`.
- Revisión: `github.com/rgap/rgap-Integracion-de-Servicios-Cloud-Gcp-Agosto2026`, carpeta `Sesion25/practica/` (`practica_bigquery_python.ipynb` y `generar_ventas.py`). DuckDB: `duckdb.org`.
- Práctica 1: mismo repositorio, carpeta `Sesion25/practica_balanceo/` (`README.md`, `server.js`, `nginx.conf` y `prueba.js`). Nginx: `nginx.org`. Node.js: `nodejs.org`.
- Material previo del curso: Sesión 24, Big Data y BigQuery.
- Google Cloud: Cloud Monitoring, Cloud Logging, Cloud Load Balancing, Cloud CDN, grupos de instancias administrados y Cloud SQL.
- Las respuestas de las preguntas están en `../respuestas.md`.